

# Kubernetes Migration Guide

---

Complete guide for migrating VMware VMs to KubeVirt on k3s/k3d using hyper2kvm.

Three deployment methods are covered:

1. **Local migration + CDI upload** — run h2kvmctl on the host, upload QCOW2 to KubeVirt
2. **In-cluster Job migration** — run h2kvmctl inside a Kubernetes pod
3. **Operator-driven migration** — declare a HyperConversion CR, operator handles everything

## Prerequisites

```
# Required tools
k3d version      # k3d v5.8+
kubectl version  # v1.31+
virtctl version  # KubeVirt CLI
docker version   # or podman
h2kvmctl --version # hyper2kvm CLI

# Cluster with KubeVirt + CDI
kubectl get kubevirt -n kubevirt # Phase: Deployed
kubectl get cdis                  # Phase: Deployed
```

## Method 1: Local Migration + CDI Upload

Run h2kvmctl locally, then upload the converted QCOW2 to KubeVirt via CDI.

### Step 1: Convert VMDK to QCOW2

```
sudo h2kvmctl --config test-confs/test-rhel88.yaml
```

The config performs: - VMDK → QCOW2 conversion with zstd compression - fstab stabilization (device paths → UUIDs) - initramfs regeneration with VirtIO drivers - Serial console injection - Libvirt domain XML generation - Smoke test (VM boots, gets IP)

## Step 2: Upload to KubeVirt via CDI

```
# Port-forward CDI upload proxy
kubectl port-forward -n cdi svc/cdi-uploadproxy 18443:443 &

# Upload via DataVolume
virtctl image-upload dv rhel8-disk \
  --namespace=rhel-vms \
  --image-path=output/rhel8.8-fixed.qcow2 \
  --size=20Gi \
  --uploadproxy-url=https://localhost:18443 \
  --insecure

kill %1 # stop port-forward
```

## Step 3: Deploy KubeVirt VM

```
kubectl apply -f test-confs/kubevirt-rhel8.8-deployment.yaml
```

This creates: Namespace, PVC, ConfigMap, VirtualMachine, SSH/VNC Services.

## Step 4: Verify

```
kubectl get vm,vmi -n rhel-vms
# NAME          STATUS    READY
# rhel8-migrated Running   True

virtctl console rhel8-migrated -n rhel-vms
```

---

## Method 2: In-Cluster Job Migration

Run the entire h2kvmctl pipeline inside a Kubernetes Job pod.

### Step 1: Build and load container image

```
# Build worker image (has qemu-img, lvm2, kpartx)
docker build --target worker -t hyper2kvm:worker -f Dockerfile .
```

```
# Load into k3d
k3d image import hyper2kvm:worker -c hyper2kvm-test
```

## Step 2: Copy VMDK into k3d node

```
# k3d nodes are Docker containers – host paths aren't directly accessible
docker exec k3d-hyper2kvm-test-agent-0 mkdir -p /tmp/hyper2kvm-input
docker cp esx8.0-rhel8.8-with-thin-provision-disk1.vmdk \
  k3d-hyper2kvm-test-agent-0:/tmp/hyper2kvm-input/
```

## Step 3: Deploy all resources

```
kubectl apply -f k8s/migration/rhel88-k3s-migration.yaml
```

This single YAML creates: - `hyper2kvm-migration` namespace - ConfigMap with `migration.yaml` config - PVCs for input (10Gi) and output (25Gi) - Job: copy VMDK from node `hostPath` into input PVC - Job: run `h2kvmctl` migration (privileged, NBD, VMCraft) - KubeVirt VM definition (Manual `runStrategy`) - SSH NodePort service

## Step 4: Monitor migration

```
# Watch jobs
kubectl get jobs -n hyper2kvm-migration -w

# Follow migration logs
kubectl logs -n hyper2kvm-migration -l step=migrate -f

# Expected output:
#   rhel88-copy-vmdk      Complete   (14s)
#   rhel88-migration     Complete   (6m48s)
```

## Step 5: Start the VM

```
kubectl patch vm rhel88-migrated -n hyper2kvm-migration \
  --type merge -p '{"spec":{"runStrategy":"Always"}}'

# Verify
kubectl get vmi -n hyper2kvm-migration
# rhel88-migrated   Running   10.42.0.78   Ready
```

## Step 6: Export libvirt domain XML

```
# Extract XML from output PVC
kubectl run extract-xml -n hyper2kvm-migration --rm -i \
  --restart=Never --image=fedora:43 \
  --overrides='{ "spec": { "containers": [ { "name": "x", "image": "fedora:43",
    "command": [ "cat", "/output/libvirt/rhel88-migrated.xml" ],
    "volumeMounts": [ { "name": "o", "mountPath": "/output", "readOnly": true } ] },
    "volumes": [ { "name": "o", "persistentVolumeClaim":
      { "claimName": "rhel88-qcow2-output" } } ] } }' > output/libvirt/rhel88-k8s.xml

# Define in libvirt
virsh define output/libvirt/rhel88-k8s.xml
virsh start rhel88-migrated
```

## Automation script

```
# One-shot: build → load → deploy → wait → start → export
./scripts/run-k8s-migration.sh

# Skip image build (if already loaded)
./scripts/run-k8s-migration.sh --skip-build

# Skip VMDK copy (if already in PVC)
./scripts/run-k8s-migration.sh --skip-build --skip-copy
```

---

## Method 3: Operator-Driven Migration

The HyperConversion operator watches CRDs and orchestrates: DataVolume creation → CDI import → VM creation.

### Step 1: Build and load operator image

```
# Build Go operator (distroless runtime)
docker build -t hyper2kvm-operator:latest -f operator/Dockerfile operator/

# Load into k3d
k3d image import hyper2kvm-operator:latest -c hyper2kvm-test
```

## Step 2: Install CRDs

```
kubectl apply -f operator/config/crd/bases/hyper2kvm.io_hyperconversions.yaml
kubectl apply -f operator/config/crd/hyper2kvm.io_validations.yaml

# Verify
kubectl get crd hyperconversions.hyper2kvm.io
```

## Step 3: Deploy operator + HyperConversion CR

```
kubectl apply -f k8s/operator-deploy/deploy-operator-migrate.yaml
```

This deploys: - `hyper2kvm-system` namespace with ServiceAccount + ClusterRole + ClusterRoleBinding - Operator Deployment (Go binary, distroless image, non-root) - HTTP file server (busybox httpd) serving the converted QCOW2 - HyperConversion CR pointing to the HTTP source URL

## Step 4: Watch the operator

```
# Operator logs
kubectl logs -n hyper2kvm-system -l control-plane=controller-manager -f

# HyperConversion status
kubectl get hc -n hyper2kvm-migration -w

# Expected progression:
#   PHASE      PROGRESS  DATAVOLUME          VM
#   Pending    0
#   Uploading   50        rhel88-operator-migration-dv
#   Converting  75        rhel88-operator-migration-dv
#   CreatingVM  75        rhel88-operator-migration-dv
#   Ready      100       rhel88-operator-migration-dv  rhel88-operator-vm
```

## Step 5: Verify VM

```
kubectl get vm,vmi -n hyper2kvm-migration
# rhel88-operator-vm  Running  True  10.42.0.93

virtctl console rhel88-operator-vm -n hyper2kvm-migration
```

## HyperConversion CR Reference

```
apiVersion: hyper2kvm.io/v1alpha1
kind: HyperConversion
metadata:
  name: my-migration
  namespace: my-namespace
spec:
  source:
    url: "http://fileserver.svc:8080/disk.qcow2" # HTTP/HTTPS URL
    checksum: "sha256:..." # Optional integrity check

  storage:
    storageClass: local-path-immediate # Must use Immediate binding for k3d
    size: 20Gi
    accessMode: ReadWriteOnce # ReadWriteOnce or ReadWriteMany
    volumeMode: Filesystem # Filesystem or Block

  vm:
    name: my-vm
    cpu:
      cores: 2 # 1-128
      sockets: 1
      threads: 1
    memory: "4Gi"
    firmware: bios # bios, uefi, or uefi-secure
    runStrategy: Always # Always, Manual, Halted, RerunOnFailure
    networks:
      - name: default
        type: pod # pod, bridge, sriov, multus

  conversion:
    compression: zstd # zstd, zlib, none
    offlineFixes: true
    timeout: 30 # minutes (5-1440)
```

## Operator Phase Lifecycle

```
Pending
└─ Creates CDI DataVolume from spec.source.url
Uploading
└─ Watches DataVolume progress (0%-100%)
└─ Updates status.uploadProgress
Converting
└─ Marks conversion complete (image already converted)
```

```
CreatingVM
  ↳ Creates KubeVirt VirtualMachine from spec.vm
  ↳ Sets owner references for cleanup
Ready
  ↳ Terminal success state (progress=100)
  ↳ VM is running

Failed
  ↳ Terminal error state
  ↳ DataVolume import failed or timeout exceeded
```

---

## k3d-Specific Notes

### Storage class for CDI

k3d's default `local-path` storage class uses `WaitForFirstConsumer` binding, which causes CDI DataVolume imports to stall. Create an Immediate binding class:

```
kubectl apply -f - <<EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-path-immediate
provisioner: rancher.io/local-path
reclaimPolicy: Delete
volumeBindingMode: Immediate
EOF
```

If the PVC stays in `Pending`, annotate it with a node:

```
kubectl annotate pvc <pvc-name> -n <namespace> \
  volume.kubernetes.io/selected-node=k3d-<cluster>-agent-0
```

### Privileged containers

Migration pods require privileged access for NBD device operations:

```
securityContext:
  runAsUser: 0
  privileged: true
```

```
capabilities:
  add: [SYS_ADMIN, MKNOD, SYS_CHROOT]
```

## /dev mount

Do NOT mount `/dev` as a hostPath volume in k3d — it causes `termination-log` conflicts with runc. Privileged containers already have `/dev` access.

## Loading images

```
# Load images into k3d (required since k3d can't pull from local Docker)
k3d image import hyper2kvm:worker -c <cluster-name>
k3d image import hyper2kvm-operator:latest -c <cluster-name>
```

## Copying files into k3d nodes

```
# k3d nodes are Docker containers
docker exec k3d-<cluster>-agent-0 mkdir -p /tmp/data
docker cp local-file.vmdk k3d-<cluster>-agent-0:/tmp/data/
```

---

# Troubleshooting

## Migration job fails with “requires root”

The worker image runs as user `hyper2kvm` by default. Add `runAsUser: 0` to the pod’s security context.

## CDI DataVolume stuck in WaitForFirstConsumer

Use `local-path-immediate` storage class or annotate the PVC with `volume.kubernetes.io/selected-node` .

## CDI webhooks blocking CR creation

If webhooks were deleted/corrupted, restart CDI pods:

```
kubectl delete pod -n cdi -l app=containerized-data-importer
```



## Operator not processing HyperConversion CR

Check that: 1. The HyperConversion controller is registered in `cmd/main.go` 2. The `v1alpha1` scheme is added to the runtime scheme 3. The operator has RBAC for `hyperconversions`, `datavolumes`, `virtualmachines`

```
kubectl logs -n hyper2kvm-system -l control-plane=controller-manager
```

## dracut fails with "Invalid tmpdir"

Inside containers, `/var/tmp` may not exist. The migration continues with the existing `initramfs` — VirtIO drivers are injected via a separate dracut invocation that creates the `tmpdir`.

## File Reference

| File                                                                      | Purpose                                     |
|---------------------------------------------------------------------------|---------------------------------------------|
| <code>test-confs/test-rhel88.yaml</code>                                  | Local migration config (VMDK→QCOW2+libvirt) |
| <code>test-confs/kubevirt-rhel8.8-deployment.yaml</code>                  | KubeVirt VM deployment (NS+PVC+VM+Services) |
| <code>k8s/migration/rhel88-k3s-migration.yaml</code>                      | In-cluster Job migration (all-in-one)       |
| <code>k8s/operator-deploy/deploy-operator-migrate.yaml</code>             | Operator deployment + HyperConversion CR    |
| <code>scripts/run-k8s-migration.sh</code>                                 | Automation script for Method 2              |
| <code>operator/config/crd/bases/hyper2kvm.io_hyperconversions.yaml</code> | HyperConversion CRD                         |
| <code>operator/controllers/hyperconversion_controller.go</code>           | Operator reconciliation logic               |

## Verified Results

All three methods tested end-to-end on k3d v5.8.3 / k3s v1.31.5 with KubeVirt v1.7.0 and CDI v1.64.0:

| Method             | Source           | Duration                   | Result                     |
|--------------------|------------------|----------------------------|----------------------------|
| Local + CDI upload | 3.9G VMDK        | ~5min convert + 47s upload | VM Running, Guest OS: RHEL |
| In-cluster Job     | 3.9G VMDK in PVC | 6m48s total                | VM Running, IP assigned    |
| Operator-driven    | QCOW2 via HTTP   | ~2min CDI import           | VM Running, IP 10.42.0.93  |